UNIVERSITÉ JOSEPH KI-ZERBO

INSTITUT BURKINABÈ DES ARTS ET MÉTIERS · IBAM

IBAM **UJKZ**

DOSSIER D'ARCHITECTURE

Capacité cible : quatre serveurs applicatifs · Exécution mesurée : deux, selon les ressources disponibles

PROJET DE GROUPE 18 · 4 MEMBRES

Master 1 Informatique · Virtualisation et Cloud Computing

Année universitaire 2025–2026 Enseignant : Dr SOMDA Flavien Hervé

ÉQUIPE DU PROJET

Nom	Prénom	Numéro	Filière
BARRY	El Nour	66806583	SI
BAMBARA	Mireille Epiphanie Joëlle Guerla	6539632	SIE
SAWADOGO	Wenden Kogondé Aimé Ephraïm	72620579	SI
WEDRAOGO	Relwende Diane Micheline	76677438	SIE

Dépôt Git : github.com/ephraim2002/resultats-bf

Document de travail daté du 27 octobre 2026 · Statut des scénarios détaillé dans le rapport

TABLE DES MATIÈRES

TABLE DES MATIÈRES 2
SYNTHÈSE EXÉCUTIVE ET EXIGENCES 3
ÉTAT DE L'ENVIRONNEMENT EXAMINÉ LE 27/10/2026 3
1. ARCHITECTURE GÉNÉRALE ET FLUX 3
2. VUE D'ENSEMBLE DE L'ARCHITECTURE 4
2. MACHINES VIRTUELLES, RESSOURCES ET RÉSEAUX 5
2.2 FLUX RÉSEAU ET PORTS 6
3. APPLICATION, API ET DONNÉES 7
4. PROVISIONNEMENT ET REPRODUCTIBILITÉ 7
4.2 IDEMPOTENCE ET GESTION DES CONFIGURATIONS 8
5. RÉPARTITION DE CHARGE ET RÉSULTATS 8
6. RÉPLICATION POSTGRESQL ET STRATÉGIE DE REPRISE 9
7. HAUTE DISPONIBILITÉ DES RÉPARTITEURS 9
8. SUPERVISION ET OBSERVABILITÉ 10
9. VALIDATION V1–V6 ET BILAN DES PREUVES 10
9.2 SÉCURITÉ, RISQUES ET LIMITES 11
10. ORGANISATION, DÉCISIONS ET PLAN DE SOUTENANCE 11
10.1 CONCLUSION ET PLAN D'ACTION DE REMISE 12
ANNEXE — PROTOCOLE DE SOUTENANCE ET RÉFÉRENCES 12

SYNTHÈSE EXÉCUTIVE ET EXIGENCES

Le projet RÉSULTATS-BF met en œuvre une plateforme permettant de retrouver un résultat d'examen à partir d'un matricule. L'objectif central n'est pas de construire une application métier complexe : il est de rendre l'infrastructure reproductible, extensible par paramétrage, observable et capable de tolérer certaines pannes.

Architecture cible : postes et hyperviseur VirtualBox ; orchestration Vagrant ; réseau privé 192.168.63.0/24 ; quatre instances applicatives Flask/Gunicorn configurables ; PostgreSQL primaire et réplica en flux ; deux répartiteurs HAProxy/Keepalived avec une adresse virtuelle ; supervision Prometheus/Grafana optionnelle. Pour respecter la mémoire disponible sur le poste de travail, l'exécution mesurée a volontairement été limitée à deux applications (NB_APP=2) ; app3/app4 sont prévues par le projet et le Vagrantfile, mais n'étaient pas démarrées lors de cette campagne.

ÉTAT DE L'ENVIRONNEMENT EXAMINÉ LE 27/10/2026

- 7 VMs indiquées « running » par Vagrant : bdd1, bdd2, app1, app2, lb1, lb2 et obs.
- Configuration effectivement exécutée : NB_APP=2, ENABLE_HA=true, ENABLE_OBS=true. Le projet accepte NB_APP de 2 à 4 ; le choix de deux instances pendant cette validation est volontaire et lié aux ressources mémoire disponibles sur le poste, non à une limite fonctionnelle du projet. app3/app4 restent des nœuds prévus mais non démarrés dans cette exécution.
- « vagrant validate » : réussite.
- Contrôles HTTP en lecture seule : API /info et /sante en HTTP 200 via localhost:8080 et via la VIP 192.168.63.10 ; les deux hôtes app1/app2 répondent directement.
- Recherche du candidat MAT000001 : HTTP 200 ; le matricule retourné correspond à la requête.
- PostgreSQL bdd1 : 50 000 lignes. PostgreSQL bdd2 : 50 000 lignes et mode standby actif.
- Réplication observée : état « streaming » ; LSN envoyé égal au LSN rejoué au relevé.
- Répartition active : sur 200 requêtes /info séquentielles à NB_APP=2, 100 réponses app1 et 100 app2 ; zéro erreur, écart 0 %, débit observé 85,8 requêtes/s. Ce relevé a été fait avec deux applications actives par choix de ressources, alors que le profil cible prévoit quatre.
- Prometheus prêt ; 10 cibles « up » sur 12 actives. Les deux cibles « down » sont les Node Exporter app3/app4:9100 : ces serveurs font partie de la capacité cible, mais ont été volontairement laissés arrêtés (NB_APP=2) pour respecter les ressources du poste.
- Grafana API health : HTTP 200, base « ok », version 13.2.4. La consultation anonyme de l'API de recherche des dashboards retourne HTTP 401 : l'interface et le tableau de bord ne sont donc pas certifiés par cette vérification.

Les mesures décrivent cette instance limitée volontairement à deux serveurs pour raison de ressources ; elles ne prouvent pas à elles seules une reconstruction intégrale, un scénario de panne, une mesure à quatre nœuds ou l'extension sans interruption. La configuration cible reste quatre applications.

1. ARCHITECTURE GÉNÉRALE ET FLUX

La publication de résultats scolaires crée un pic de consultation qui peut révéler une capacité insuffisante ou un point de défaillance unique. Le projet consiste à fournir une infrastructure décrite sous forme de code, reproductible sur un poste de travail et extensible sans modifier manuellement le fichier de répartition.

Les exigences essentielles du sujet joint sont :

R1 — reconstruction par Vagrant sans étape manuelle après le provisionnement ;

R2 — provisionnement réexécutable et idempotent ;

R3 — dimensionnement applicatif paramétrable ;
R4 — sonde de santé servant au retrait des instances défaillantes ;
R5 — réplication PostgreSQL ;
R6 — procédure de promotion documentée ;
R7 — secrets extérieurs au dépôt ;
R8 — trafic applicatif interne sur réseau privé, entrée via le répartiteur.

Le sujet demande des mesures V1 à V6, des extensions, un dossier d'architecture de 10 à 15 pages et un journal de 1 à 2 pages. Le présent dossier décrit la cible et l'état constaté le 27 octobre 2026 ; il différencie résultat vérifié, trace historique et test restant.

Hors périmètre du prototype : exposition publique sur Internet, comptes utilisateurs, chiffrement TLS et promotion automatique de la base. La cible applicative comprend quatre serveurs ; la campagne documentée n'en a exécuté que deux, choix délibéré pour tenir compte de la mémoire disponible sur le poste. Le Vagrantfile permet de modifier NB_APP jusqu'à quatre ; la comparaison mesurée NB_APP=2/NB_APP=4 reste à réaliser sur un hôte doté de ressources suffisantes.

Méthode de lecture : les éléments issus du code sont des comportements prévus ; seules les commandes et mesures explicitement relevées sont rapportées comme faits. Les tests qui interrompent ou détruisent des VMs sont listés comme travaux à autoriser, et non comme succès.

2. VUE D'ENSEMBLE DE L'ARCHITECTURE

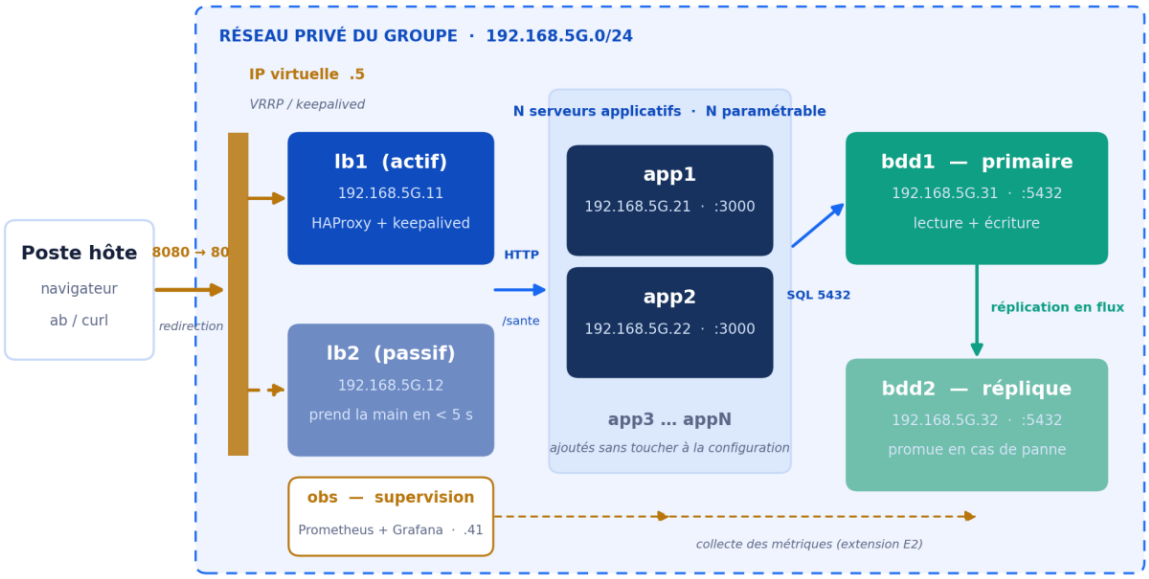
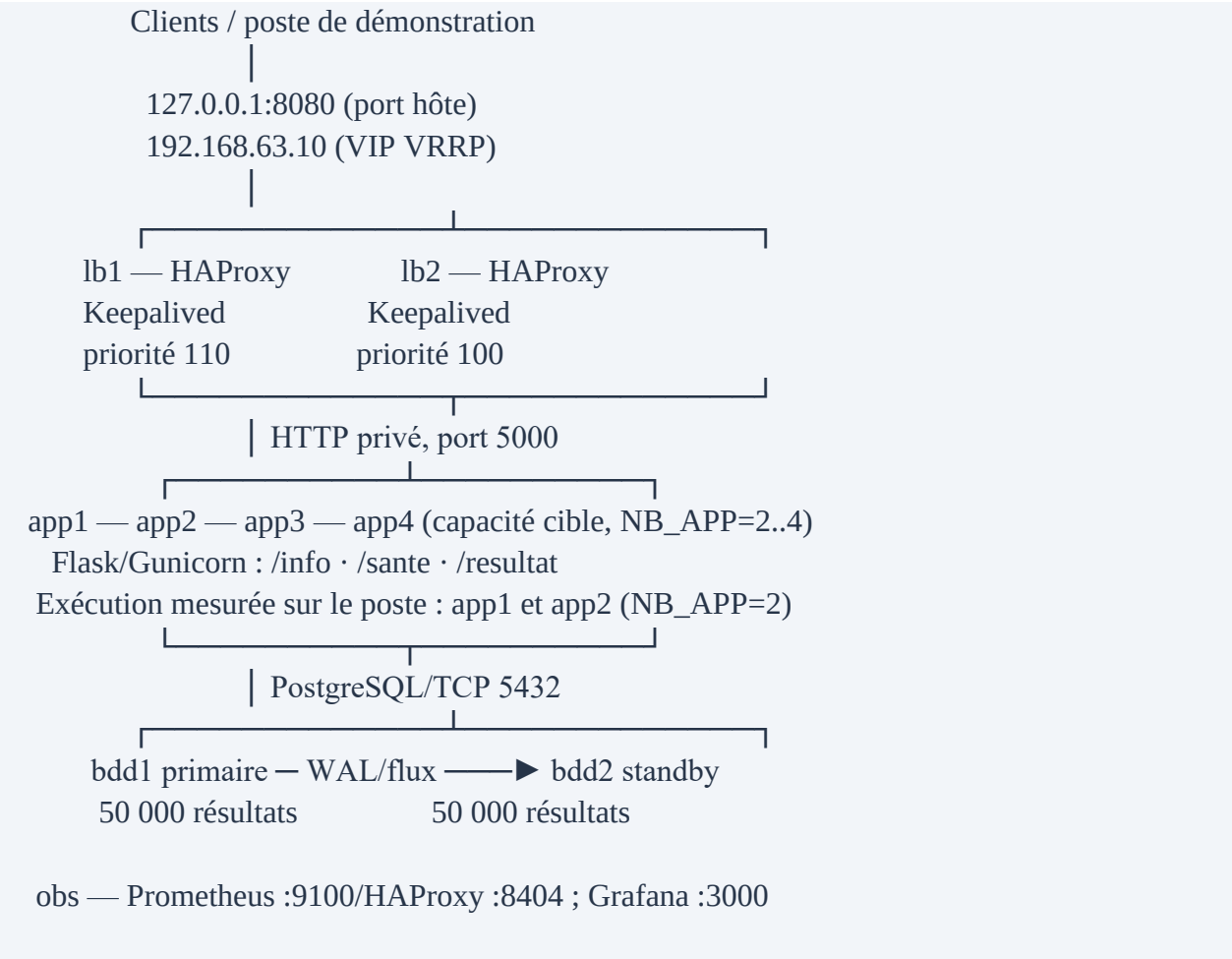


Figure 1 — Architecture cible fournie dans le cahier des charges du projet.



Le schéma logique ci-dessus distingue le trafic utilisateur, la réplication des données et la collecte de métriques. La VIP est portée par le répartiteur actif. Le forwarded port Vagrant 8080 correspond à l'accès direct à lb1 et n'est pas un substitut à la VIP pour un test de basculement.
Sources : Vagrantfile, scripts/setup-app.sh, scripts/setup-lb.sh, scripts/setup-lb-ha.sh, scripts/setup-database.sh, scripts/setup-replica.sh et scripts/setup-observability.sh. Les rôles sont déployés par des scripts Shell exécutés dans les invités Linux.

2. MACHINES VIRTUELLES, RESSOURCES ET RÉSEAUX

VM	Adresse	RAM configurée	Rôle	État au relevé
bdd1	192.168.63.31	768 Mio	PostgreSQL primaire	running
bdd2	192.168.63.32	768 Mio	PostgreSQL standby	running
app1	192.168.63.21	512 Mio	API applicative	running
app2	192.168.63.22	512 Mio	API applicative	running
lb1	192.168.63.11	512 Mio	HAProxy + Keepalived	running, port hôte 8080
lb2	192.168.63.12	512 Mio	HAProxy + Keepalived	running, port hôte 8081

VM	Adresse	RAM configurée	Rôle	État au relevé
obs	192.168.63.41	1024 Mio	Prometheus + Grafana	running

Somme mémoire attribuée pour cette configuration exécutée à deux applications : 4 608 Mio (4,5 Gio), hors système hôte. Avec app3 et app4, le total configuré serait 5 632 Mio (5,5 Gio). Le sujet donne environ 3 Gio pour son socle de cinq VM. Ici, bdd1 + bdd2 + 2 apps totalisent 2 560 Mio ; lb1/lb2 ajoutent 1 024 Mio et obs ajoute 1 024 Mio, soit 4 608 Mio. Les deux VM applicatives supplémentaires ajouteraient encore 1 024 Mio. L'équipe a donc volontairement limité le nombre d'applications actives à deux pour préserver la mémoire du poste ; le dimensionnement applicatif cible reste quatre. Ces sommes représentent la mémoire virtuelle configurée, pas une mesure de consommation RAM effectivement observée.

Dans Vagrantfile, NB_APP est borné de 2 à 4 ; ENABLE_HA ajoute lb2 et ENABLE_OBS ajoute obs. Les quatre applications peuvent être démarrées sur une machine de capacité suffisante sans changer leur code ni la configuration fonctionnelle du répartiteur. Le contrôle courant porte sur les deux applications actives, non sur un pool de quatre.

Configuration mesurée à l'exécution : NB_APP=2, ENABLE_HA=true, ENABLE_OBS=true. Le groupe a choisi deux applications pour limiter la consommation du poste de travail, tout en conservant quatre comme capacité cible. La RAM configurée par VM applicative est de 512 Mio : app1/app2 représentent 1 Gio attribué ; app3/app4 ajouteraient 1 Gio. Le choix d'exécution à deux ne signifie pas que le besoin cible est de deux serveurs. Chaque VM est configurée avec un vCPU ; les ressources effectivement disponibles côté hôte n'ont pas fait l'objet d'un relevé de saturation.

Pour reproduire la configuration mesurée, copier .env.exemple vers .env et régler NB_APP=2. Sur une machine plus puissante, sélectionner NB_APP=4 pour utiliser la capacité applicative complète du Vagrantfile. Conserver les autres paramètres identiques lors d'une comparaison.

2.2 FLUX RÉSEAU ET PORTS

Origine	Destination	Port	Utilité
Poste hôte	redirection Vagrant lb1	8080 → 80	Entrée de démonstration
Poste hôte	redirection Vagrant lb2	8081 → 80	Accès direct de secours
Client du réseau privé	VIP 192.168.63.10	80	Adresse stable Keepalived
HAProxy	app1…appN	5000	Requêtes HTTP applicatives
applications	bdd1	5432	Requêtes PostgreSQL et résultats
bdd2	bdd1	5432	pg_basebackup et streaming WAL
Prometheus	Node Exporter	9100	Métriques système
Prometheus	HAProxy	8404	Compteurs HTTP, état des backends

Origine	Destination	Port	Utilité
Navigateur hôte	Grafana / Prometheus	3000 / 9090	Interfaces et API de supervision

HAProxy écoute sur 80 dans les VM lb et expose un front-end métriques séparé. La configuration d'équilibrage suit un algorithme round-robin et interroge GET /sante ; elle attend le statut 200 et déclare indisponible une cible après deux échecs consécutifs avec une seconde entre vérifications.
Le code Flask vérifie PostgreSQL et renvoie un objet JSON de statut. La version déployée au relevé répond /sante=200 lorsque SELECT 1 aboutit. L'endpoint /info fournit l'identité du nœud sans solliciter la base ; l'endpoint /resultat lit un matricule paramétré. Le trafic inter-machine prévu emprunte le réseau privé Vagrant.

3. APPLICATION, API ET DONNÉES

Table publique resultats : matricule (clé primaire), nom, serie, moyenne, admis. Un index secondaire sur serie est déclaré dans schema.sql. Le script seed.sql peuple la table de 50 000 lignes déterministes et tolère la réexécution par ON CONFLICT DO NOTHING.
L'utilisateur applicatif PostgreSQL doit lire les résultats mais n'a pas besoin de privilège d'écriture sur les dossiers des candidats. Les paramètres de recherche sont fournis via les paramètres liés psycopg2, non concaténés dans la requête. La connexion impose un délai de deux secondes dans l'implémentation Flask.

Méthode et route	Fonction	Réponse attendue
GET /sante	Interroge PostgreSQL avec SELECT 1	200 si accessible, 503 si inaccessible
GET /info	Identifie le serveur qui a répondu	200 et hote + service
GET /resultat?matricule=…	Lit une ligne avec paramètre SQL	200, 400 si absent, 404 si inconnu, 503 si SQL indisponible
GET /	Décrit le service	200 et index de routes

Preuve fonctionnelle relevée : /resultat?matricule=MAT000001 a retourné HTTP 200 avec matricule MAT000001, nom Eleve_000001, série C, moyenne 5,37 et admis=false. La donnée est publiée ici parce qu'il s'agit de la ligne de jeu de test synthétique, non d'une identité réelle.
La plateforme est conçue sans session métier locale : plusieurs processus applicatifs peuvent répondre et l'état de référence est PostgreSQL. Le schéma n'inclut pas de données personnelles réelles.

4. PROVISIONNEMENT ET REPRODUCTIBILITÉ

Vagrant charge .env à l'exécution ; les secrets doivent être remplacés localement et le fichier demeure exclu du dépôt. L'option synchronisée /vagrant rend les scripts, SQL, YAML et application disponibles à l'invité.
Ordre recommandé :

1. bdd1 configure PostgreSQL, rôles, schema.sql et seed.sql ;
2. bdd2 attend bdd1, initialise pg_basebackup et active le standby ;

3. app1…appN installent Python/venv/dépendances et écrivent leur service systemd, avec N réglé par NB_APP (2 à 4) ;
4. lb1 et, si ENABLE_HA=true, lb2 installent HAProxy/Keepalived ;
5. obs installe Prometheus, Grafana et leurs configurations si requis.

Le provisionnement testé sur la machine du groupe utilise N=2 par contrainte de ressources disponibles ; app3/app4 sont configurables en réglant NB_APP à 3 ou 4 sur un hôte disposant de mémoire suffisante. Il s'agit d'un compromis de la campagne, pas d'une réduction de la capacité cible.

Commandes types depuis PowerShell, après préparation d'un fichier .env local :

```
vagrant validate
vagrant up bdd1
vagrant up bdd2
# Dans .env, NB_APP=2 : profil de mesure sur le poste du groupe
vagrant up app1 app2
vagrant up lb1 lb2
vagrant up obs
```

Sur un hôte plus puissant, régler NB_APP=4 dans .env, puis démarrer app1 app2 app3 app4 avant les répartiteurs.

La reconstruction complète vagrant destroy -f est destructive pour les VMs et leurs données locales ; elle est uniquement recommandée comme test contrôlé avec sauvegarde et accord de l'équipe. La présente validation n'a pas détruit ni recréé les machines.

Le fichier setup-app.sh installe les paquets et dépendances, déploie le module, crée un env-file à permission 600, démarre resultats-api et vérifie /info pendant un délai borné. Le test de lancement ne requiert pas PostgreSQL car /info est autonome ; la sonde /sante vérifie ensuite la connexion.

Validation de syntaxe observée : vagrant validate a renvoyé « Vagrantfile validated successfully. »

4.2 IDEMPOTENCE ET GESTION DES CONFIGURATIONS

L'idempotence recherchée signifie que rejouer un provisionnement ne doit pas accumuler de règles ni provoquer d'échec pour un service déjà installé. Les scripts reconstruisent certains fichiers de configuration et activent les services systemd.

Contrôle actuel : la liste de VMs déclarées et les services indiqués en état running sont observés. En revanche, nous n'avons pas effectué dans cette passe un double vagrant provision successif avec calcul de sommes avant/après ; R2 est donc conçu par les scripts mais n'est pas re-mesuré ici.

Point d'attention opérationnel : plusieurs scripts Debian installent des paquets par apt, et les scripts d'initialisation des bases diffèrent dans leur stratégie. Avant qualification, exécuter les scripts deux fois et comparer les versions/permissions/hash des fichiers réellement gérés.

Déploiement applicatif : le script de provisionnement remet le code en place puis redémarre systemd. Le test de santé après le redémarrage est un test de readiness HTTP, pas un test de démarrage parfait de l'ensemble du parc.

Recommandation de contrôle : inventorier chaque fichier géré par script, écrire dans un fichier temporaire, ne remplacer la cible que si le contenu change, puis redémarrer le service uniquement sur changement (cas déjà appliqué à plusieurs réglages PostgreSQL dans l'ancienne variante bdd.sh). Documenter la variante réellement exécutée avant fusion.

5. RÉPARTITION DE CHARGE ET RÉSULTATS

Mesure fraîche (poste Windows, 27/10/2026) : 200 appels séquentiels GET /info à http://127.0.0.1:8080. Chronomètre global : 2 331 ms ; réponses : app1=100, app2=100 ; erreurs réseau=0 ; débit calculé : 200 / 2,331 = 85,8 requêtes/s. Écart = 100 × (100−100)/200 = 0 % si le critère compare l'écart absolu au volume total. La grille v1_repartition.py calcule plutôt

(max−min)/N ; sur l'échantillon pair à 200 et 2 serveurs cette valeur est 0, mais sur l'historique ancien elle vaut 191/385=49,6 %. Résultat : l'essai exact de 20 appels visé par le sujet n'a pas été retesté ; 200 appels équitablement distribués ont été mesurés sur deux des quatre serveurs applicatifs configurables. Cela prouve la répartition sur la configuration disponible, pas les performances à quatre nœuds.

Ce débit porte sur /info, réponse légère n'interrogeant pas la base, sur une série séquentielle, et n'est pas comparable à une campagne ApacheBench concurrente. Aucun résultat ne doit être extrapolé à des dizaines de milliers de clients.

Historique fourni avec le dépôt : tests/resultats/v2-20261027-195118.csv contient un scénario V2 précédent, apparemment étalé sur environ 60 secondes. Le parseur CSV retrouve 387 enregistrements ; 385 réponses possèdent un code HTTP 200 et un JSON lisible, dont app1=97 et app2=288. Deux lignes ne contiennent pas de JSON complet (à t=29,72 s sans réponse et à t=32,01 s HTTP 502). Sur les 385 enregistrements interprétables : 6,42 réponses/s environ et écart de répartitions 49,6 % du total. Ce fichier n'est PAS un benchmark « propre sans erreur » et il ne satisfait pas le seuil d'équilibrage 20 %. Il est conservé comme trace du comportement dégradé historique, pas comme résultat actuel.

La métrique officielle Prometheus/HAProxy donne des compteurs cumulatifs ; pour un débit, utiliser la variation rate(...[1m]) après génération de trafic, non le compteur brut. Aucun gain de débit à quatre serveurs n'est affirmé : il reste à le mesurer sur un hôte avec les ressources adaptées.

6. RÉPLICATION POSTGRESQL ET STRATÉGIE DE REPRISE

bdd1 héberge la base primaire. Les écritures WAL sont transmises à bdd2 par réplication physique PostgreSQL. La vérification en lecture seule montre pg_stat_replication.state=streaming ; le LSN transmis et rejoué était identique à l'instant de capture. Chaque instance détenait 50 000 lignes au moment du contrôle.

Le standby reste en lecture seule jusqu'à promotion. scripts/failover-db.sh demande explicitement la chaîne PROMOUVOIR-BDD2, contrôle que PostgreSQL est en récupération, lance pg_promote() et attend au plus 30 secondes la sortie du mode récupération. L'opération n'a pas été déclenchée pendant la présente campagne afin de préserver l'environnement.

Limites connues : la promotion est manuelle, aucune orchestration automatique du retour primaire n'est présente, et les applications ont besoin que DB_HOST soit redirigé vers le nouveau primaire. Il est impératif de vérifier quelle copie possède le WAL le plus récent avant promotion, de protéger la bascule contre les doubles primaires et d'écrire la procédure de retour/seed-back avant un déploiement de production.

Résultat vérifié : réplica en récupération et nombre de lignes identique. Résultat non vérifié : temps de bascule, perte de transactions en cours et reprise automatique de l'API après bascule.

7. HAUTE DISPONIBILITÉ DES RÉPARTITEURS

lb1 et lb2 exécutent HAProxy et Keepalived. L'instance VRRP de lb1 a la priorité 110 ; celle de lb2 100. Keepalived supervise le processus HAProxy et partage une adresse virtuelle 192.168.63.10. Lorsqu'un maître devient indisponible, l'objectif est de transférer la VIP au pair et de continuer à router le trafic HTTP.

Constat non perturbateur : la VIP 192.168.63.10 est présente sur l'interface privée d'un répartiteur au moment du relevé. L'URL de la VIP a répondu HTTP 200 à /info et /sante.

La vérification précédente n'a pas arrêté lb1 et ne mesure donc pas le délai de failover. Pour prouver E1/V6, prévoir une fenêtre contrôlée, envoyer des sondes horodatées en boucle, exécuter vagrant halt lb1, constater l'adresse VIP sur lb2, relever le premier HTTP 200 sans interruption et redémarrer lb1. Récupérer ensuite la VIP sur son état préféré uniquement si la configuration/stratégie le prévoit.

Le port 8080 redirige l'hôte directement vers lb1 et peut devenir inaccessible en cas d'arrêt de lb1. Pendant le test de bascule, utiliser l'adresse VIP, depuis une machine réellement connectée au réseau hôte-seul.

8. SUPERVISION ET OBSERVABILITÉ

La VM obs héberge Prometheus et Grafana. Prometheus collecte ses propres métriques, les Node Exporter et l'endpoint HAProxy de lb1/lb2 à intervalles de 15 secondes. Le tableau Grafana du dépôt porte les vues de débit applicatif, état de backends, CPU, mémoire et disponibilité des scrapes.

Le script scripts/setup-observability.sh configure neuf cibles Node Exporter (bdd1, bdd2, app1–app4, lb1, lb2 et obs), deux endpoints HAProxy et Prometheus lui-même. Au relevé du 27/10/2026, Prometheus présentait 12 cibles actives : 10 « up » et 2 « down ». Les indisponibles étaient 192.168.63.23:9100 et 192.168.63.24:9100 (« no route to host »), correspondant à app3/app4, nœuds prévus pour la capacité cible à quatre applications, mais volontairement non démarrés dans cette exécution limitée à NB_APP=2 pour tenir compte des ressources du poste. Les 10 cibles joignables sont cohérentes avec sept Node Exporter joignables, les deux répartiteurs et Prometheus local. Ces deux états down ne signifient donc pas que le projet est limité à deux serveurs ; ils reflètent le dimensionnement choisi pour la campagne. Pour éviter les alertes prévisibles, la liste statique de cibles peut être générée selon NB_APP.

Grafana API health a répondu database=ok, version=13.2.4. La consultation anonyme d'API search a reçu HTTP 401 ; l'écran / renvoie toutefois le document Grafana. Ce contrôle de santé prouve que le processus sert son endpoint health, mais ne prouve pas la lisibilité des panneaux avec une session utilisateur. Le tableau de bord reste à vérifier avec les identifiants fournis localement.

Au relevé Prometheus, obs avait RAM totale d'environ 961,7 MiB et load average à une minute de 0,06. Ce sont des instantanés non représentatifs d'une campagne de saturation.

9. VALIDATION V1–V6 ET BILAN DES PREUVES

ID	Scénario / critère du sujet	Preuve disponible	Statut responsable
V1	20 appels /info ; équilibre <20 %	Mesure nouvelle à 200 requêtes : 2 nœuds, 100/100, 0 erreur, écart 0 %	Validé sur échantillon 200, pas 20
V2	Arrêt app1 sous charge ; dégradation <5 s	Trace CSV historique : 387 lignes, 2 échecs, dont HTTP 502 ; la baisse de répartition commence vers 29,72 s et les 200 réapparaissent vers 32,33 s	Partiel ; CSV suggère une perturbation d'environ 2,6 s, mais ne consigne pas la commande d'arrêt, le retour HTTP brut ni l'heure de reprise. Il ne certifie donc pas le délai de panne
V3	`ab -n 2000 -c 50`, comparer NB_APP=2 puis 4	200 requêtes séquentielles mesurées à NB_APP=2 ; pas de campagne concurrente	Partiel ; refaire sur un hôte avec mémoire suffisante pour 4 applications

ID	Scénario / critère du sujet	Preuve disponible	Statut responsable
		comparative 2/4	
V4	arrêt bdd1, bascule sur bdd2, mesurer durée	Réplica streaming contrôlé ; failover non déclenché	Non testé, opération destructive
V5	destroy puis up sur poste vierge	`vagrant validate` et parc actuel running	Non validé ; aucune destruction exécutée
V6/E1	arrêter lb1, conserver l'accès VIP	VIP observée présente et répond avant arrêt	Non testé après panne

Les cases « à exécuter » sont des actions réservées à une fenêtre de test, sauvegardes et autorisation de l'équipe. La présence du script d'essai n'équivaut pas à un verdict positif.

9.2 SÉCURITÉ, RISQUES ET LIMITES

- Identifiants DB : .env hors Git, fichier d'environnement Linux protégé par chmod 600. Vérifier systématiquement git status avant partage. Ne jamais inclure les valeurs de secrets dans les captures/journaux/livrables.
- API sans TLS : les ports réseau privé ne chiffrent pas le trafic. Usage pédagogique sur hôte isolé, non configuration Internet.
- Sécurité des sorties d'erreur : renvoyer un code JSON générique et journaliser les détails serveur sans exposer un DSN ou un mot de passe.
- Base unique en écriture : le streaming replica réduit l'exposition à la perte disque mais ne fournit pas de quorum automatique.
- Dépendance au poste : un seul ordinateur et un hyperviseur local partagent processeur, mémoire, disque et alimentation ; il ne s'agit pas de plusieurs domaines de panne indépendants.
- Supervision : deux scrapes app3/app4 sont en erreur car ces serveurs prévus sont volontairement arrêtés dans la campagne NB_APP=2, choisie en raison de la mémoire du poste. La capacité cible reste quatre applications. Générer les cibles Node Exporter selon NB_APP éviterait ces alertes attendues.
- Manque de mesures V3/V5 : ne pas annoncer amélioration de capacité sans résultat contrôlé.
- CSV historique : 2 lignes illisibles/de statut non-200 ; cette trace n'est pas un succès E4.
- Logos : les deux fichiers officiels ne sont pas disponibles dans le code et le sujet joint ne contient pas d'image sur sa couverture. Les zones réservées doivent être remplacées par les fichiers officiels autorisés avant envoi.

10. ORGANISATION, DÉCISIONS ET PLAN DE SOUTENANCE

Les quatre étudiants listés en page de garde constituent le groupe responsable du rendu. Le cahier de sujet impose quatre domaines (réseau/répartition, applicatif, données, essais/observabilité), mais les documents du dépôt ne permettent pas d'attester avec certitude la personne affectée à chacun. Le tableau à compléter ci-dessous empêche de créditer un rôle individuel sans confirmation du groupe.

Domaine du projet	Contribution constatée dans le dépôt	Responsable nominatif à confirmer

Domaine du projet	Contribution constatée dans le dépôt	Responsable nominatif à confirmer
Réseau / Vagrant / HAProxy / Keepalived	Vagrantfile, configurations et scripts de répartition	À compléter par le groupe
Application / API / E4	Flask, scripts d'installation et déploiement progressif	À compléter par le groupe
Données / réplication / failover	PostgreSQL, schéma, seed, réplica et script failover	À compléter par le groupe
Tests / observabilité / rapport	test de répartition, charge continue, CSV, Prometheus/Grafana	À compléter par le groupe

Décisions structurantes : Vagrant rend déclaratifs les nœuds ; NB_APP règle le nombre d'applications ; les IP de nœuds répondent au sous-réseau du groupe 18 ; PostgreSQL centralise l'état ; HAProxy sépare un point d'entrée logique des instances applicatives ; Keepalived partage une VIP ; Prometheus collecte les mesures. Ces choix sont cohérents avec la résilience et la scalabilité pédagogique, tout en gardant explicites leurs points de défaillance locaux.

10.1 CONCLUSION ET PLAN D'ACTION DE REMISE

L'environnement examiné fournit un socle fonctionnel observable : Vagrant a validé la syntaxe ; sept machines sont indiquées running dans le relevé avec NB_APP=2 ; l'API répond sur le port local et via la VIP ; sur 200 requêtes séquentielles /info, app1 et app2 ont répondu 100 fois chacune sans erreur ; la base et sa réplique contiennent 50 000 lignes et le flux est en streaming. Le Vagrantfile permet une capacité cible de quatre applications, mais l'hôte du groupe n'en a exécuté que deux pour des raisons de ressources. Prometheus est prêt : 10 cibles sur 12 sont joignables. Grafana health répond, mais l'accès anonyme à l'API de recherche de dashboards retourne 401 ; l'affichage des panneaux n'a donc pas été validé.

Les preuves ne doivent pas être surinterprétées : V1 est vérifié sur 200 appels, et non sur les 20 appels de l'épreuve ; V2 repose sur un CSV historique contenant deux échecs, sans preuve synchronisée de l'arrêt d'une VM ; V3, la promotion de la base, l'arrêt de lb1 et la reconstruction vierge ne sont pas démontrés. Ces scénarios ne sont pas déclarés réussis.

Avant l'envoi au professeur, l'équipe doit vérifier les rôles nominatifs, insérer les logos institutionnels autorisés, vérifier l'URL du dépôt, contrôler les secrets sans publier leurs valeurs et, si une fenêtre de laboratoire est disponible, compléter les scénarios non prouvés avec sauvegarde, autorisation et rapports horodatés. Les observations datées du 27 octobre 2026 décrivent un environnement pédagogique et ne constituent pas un certificat de production.

ANNEXE — PROTOCOLE DE SOUTENANCE ET RÉFÉRENCES

Répartition non destructive : python tests/v1_repartition.py http://127.0.0.1:8080 20 2 ; ou tests/validation-finale.ps1 -Samples 200 si aucune autre démonstration ou charge n'utilise l'environnement. Conserver le CSV/JSON produit.

Charge : ab -n 2000 -c 50 http://127.0.0.1:8080/info, uniquement en campagne autorisée. Relever débit, latence p95, erreurs et ressources ; comparer NB_APP=2 et NB_APP=4 dans des conditions identiques. Le test NB_APP=4 doit être réalisé sur un hôte capable d'allouer environ 5,5 Gio aux VM configurées, en plus de la mémoire nécessaire au système hôte.

Pannes : pour V2, arrêter une application ; pour V6, tester la VIP http://192.168.63.10 après arrêt contrôlé de lb1 ; pour V4, promouvoir bdd2 seulement après sauvegarde et accord explicite ; pour V5, reconstruire sur un poste de test vierge. Pour chacun, horodater commande, codes HTTP, retour stable, LSN/compte SQL et état des VM. Ces actions peuvent interrompre le service ou effacer les disques virtuels ; elles ne sont pas à lancer sans autorisation.

Pendant toute campagne, noter date, versions, variables NB_APP/ENABLE_HA/ENABLE_OBS (sans secrets), état des VMs, URL testée, commandes, durée, erreurs et limites. Ne jamais inclure de mot de passe ni de données personnelles dans les pièces remises.

Fichiers de référence : Vagrantfile, .env.exemple (modèle seulement), scripts/setup-app.sh, scripts/setup-database.sh, scripts/setup-replica.sh, scripts/setup-lb-ha.sh, scripts/failover-db.sh, scripts/setup-observability.sh, app/app.py, database/schema.sql, database/seed.sql, tests/validation-finale.ps1, tests/v1_repartition.py, tests/resultats/v2-20261027-195118.csv et le tableau Grafana.